

Assignment 9.1 Built-In Objects and Advanced Function Usage	
Course Code: CPE 025 B	Program: BSCPE
Course Title: Software Design 2	Date Performed: 03/07/2026
Section: CPE32S4	Date Submitted: 03/07/2026
Name: Bautista, Jhon Hendricks	Instructor: Engr. Neal Barton James Matira

3. Procedure

Accomplish the following in JSE 2: Module 3 and 4:

- Module 3
 - LAB Code Challenge #1
 - LAB Code Challenge #2
 - LAB Code Challenge #3
 - LAB Code Challenge #4
 - LAB Code Challenge #5
- Module 4
 - LAB Code Challenge #1
 - LAB Code Challenge #2
 - LAB Code Challenge #3
 - LAB Code Challenge #4
 - LAB Code Challenge #5

Provide an Analysis for each LAB Code Challenge.

Use the provided [Lab Report Template](#) to document your activity.

4. Output

MODULE 3

LAB Code Challenge #1

```
function getRandomSet(m, n, uniq, sorted) {
  let retVal = uniq ? new Set() : [];
  let push = (e) => uniq ? retVal.add(e) : retVal.push(e);
  let length = () => uniq ? retVal.size : retVal.length;

  for(;length() < m; ) {
    push(Math.floor(Math.random() * Math.floor(n)));
  }

  return sorted ? [...retVal].sort((n1, n2) => n1 - n2) : [...retVal];
}

console.log(getRandomSet(10, 20, false, false));
console.log(getRandomSet(10, 20, false, true));
console.log(getRandomSet(10, 20, true, false));
console.log(getRandomSet(10, 20, true, true));
```

Figure 1. Program

In figure 1 it shows the program for the challenge 1 wherein I used the function getRandomSet function which is mainly for generating a random set of numbers based on the passed parameters. This function can create a set of numbers

that has unique or duplicates or sorted or not sorted due to the value of the uniq and sorted parameters being used in the logical operator which checks the value of the randomly generated number.

▼ Array(10) [challenge1.js:13](#)
0: 6
1: 13
2: 10
3: 14
4: 14
5: 16
6: 7
7: 12
8: 17
9: 15
length: 10
▶ [[Prototype]]: Array(0)

▼ Array(10) [challenge1.js:14](#)
0: 2
1: 2
2: 3
3: 4
4: 5
5: 12
6: 13
7: 16
8: 17
9: 18
length: 10
▶ [[Prototype]]: Array(0)

▼ Array(10) [challenge1.js:15](#)
0: 5
1: 10
2: 8
3: 14
4: 19
5: 11
6: 1
7: 0
8: 2
length: 10
▶ [[Prototype]]: Array(0)

▼ Array(10) [challenge1.js:16](#)
0: 2
1: 4
2: 5
3: 7
4: 8
5: 10
6: 11
7: 13
8: 14
9: 16
length: 10
▶ [[Prototype]]: Array(0)

Figure 2. Output

Based on the output in figure 2 the program can successfully generated non-unique, sorted or not sorted numbers shown in the first two generated list.

Then the function also successfully shown unique, sorted and not sorted numbers.

LAB Code Challenge #2

JavaScript

```
let nameRegExp = /^[A-Z][a-z]+$/;
let emailRegExp =
  /^([a-zA-Z]+\.\.)*[a-zA-Z]+@([a-zA-Z]+\.\.)*[a-zA-Z]{2,3}$/;

class User {
  #name;
  #surname;
  #email;

  constructor(name, surname, email) {
    this.name = name;
    this.surname = surname;
    this.email = email;
  }

  get name() {
    return this.#name;
  }
  set name(val) {
    if (typeof val === 'string' && val.match(nameRegExp)) {
      this.#name = val;
    } else {
      throw(new Error(`Incorrect name format: ${val}`));
    }
  }

  get surname() {
    return this.#surname;
  }
  set surname(val) {
    if (typeof val === 'string' && val.match(nameRegExp)) {
      this.#surname = val;
    } else {
      throw(new Error(`Incorrect surname format: ${val}`));
    }
  }

  get email() {
    return this.#email;
  }
  set email(val) {
    if (typeof val === 'string' && val.match(emailRegExp)) {
      this.#email = val;
    } else {
      throw(new Error(`Incorrect email format: ${val}`));
    }
  }
}
```

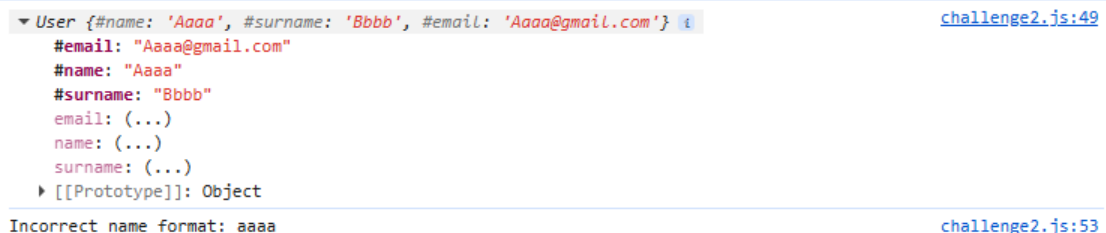
```

try {
  let user1 = new User('Aaaa', 'Bbbb', 'Aaaa@gmail.com');
  console.log(user1);
  let user2 = new User('aaaa', 'Bbbb', 'Aaaa@gmail.com'); // -> Error
} catch(err) {
  console.log(err.message);
}

```

Figure 3. Program

Based on the program in figure 3 this mainly uses the User class which creates the persons name, surname and email. This program also utilizes regex in order to validate the valid characters for the input name which is in the set name function. Then it also has a error handling shown in the try catch block wherein it passes the data to the User class and tries to create a object and then it outputs an error message if the input was not valid.



```

▼ User {#name: 'Aaaa', #surname: 'Bbbb', #email: 'Aaaa@gmail.com'} challenge2.js:49
  #email: "Aaaa@gmail.com"
  #name: "Aaaa"
  #surname: "Bbbb"
  email: (...)
  name: (...)
  surname: (...)
  ► [[Prototype]]: Object
Incorrect name format: aaaa challenge2.js:53

```

Figure 4. Output

The output in figure 4 shows that the first input was valid resulting in the data being printed while the second input object caused the error message since the input name for the user was not valid according to the regex check.

LAB Code Challenge #3

```

class Users {
  #users;

  constructor() {
    this.#users = new Map();
  }

  add(name, surname, email) {
    try{
      this.#users.set(email, new User(name, surname, email));
    } catch(e) {
      console.log(e.message);
    }
  }

  delete(email) {
    (parameter) email: any
    return this.#users.delete(email);
  }

  get(email) {
    return this.#users.get(email);
  }

  getAll(sortBy) { //name,surname,email
    return [...this.#users].sort((u1,u2) => u1[1][sortBy] > u2[1][sortBy] ? 1 : -1).map(u => u[1]);
  }
}

let users = new Users();
users.add("Aaaa", "Bbbb", "cccc@gmail.com");
users.add("Mmmm", "Ffff", "eeee@gmail.com");
users.add("Aaaa", "Bbbb", "cccc@gmail.com");
users.add("Xxxx", "Oooo", "dddd@gmail.com");
console.log(users.get("dddd@gmail.com"));
console.log(users.getAll("name").map(u => u.name));
console.log(users.getAll("surname").map(u => u.surname));
console.log(users.getAll("email").map(u => u.email));

```

Figure 5. Program

In figure 5 it has the similar structure of the previous challenge program but now it uses a map collection for managing the objects created in the User class. It has functions for adding, adding, deleting, retrieving, and sorting users. The map collection utilizes the key value structure in which this program utilizes by having a key for the object attributes and the value being the passed value to the constructor.

```

challenge3.js:80
▼ User {#name: 'Xxxx', #surname: 'Oooo', #email: 'ddd@gmail.com'} ⓘ
  #email: "ddd@gmail.com"
  #name: "Xxxx"
  #surname: "Oooo"
  email: (...)
  name: (...)
  surname: (...)
  ▶ [[Prototype]]: Object

challenge3.js:81
▼ (3) ['Aaaa', 'Mmmm', 'Xxxx'] ⓘ
  0: "Aaaa"
  1: "Mmmm"
  2: "Xxxx"
  length: 3
  ▶ [[Prototype]]: Array(0)

challenge3.js:82
▼ (3) ['Bbbb', 'Ffff', 'Oooo'] ⓘ
  0: "Bbbb"
  1: "Ffff"
  2: "Oooo"
  length: 3
  ▶ [[Prototype]]: Array(0)

challenge3.js:83
▼ (3) ['cccc@gmail.com', 'ddd@gmail.com', 'eee@gmail.com'] ⓘ
  0: "cccc@gmail.com"
  1: "ddd@gmail.com"
  2: "eee@gmail.com"
  length: 3
  ▶ [[Prototype]]: Array(0)

```

Figure 6. Output

Based on the output in figure 6 the program is able to successfully map the object's attributes.

LAB Code Challenge #4

```

JavaScript
class Point {
  constructor(x, y) {
    this.type = 'point';
    this.x = x;
    this.y = y;
  }
}

class Line {
  constructor(points) { // [[x, y], [], [], ...]
    this.type = 'line';
    this.points = points.map(p => new Point(p[0], p[1]));
  }
}

```

```

class Figure {
  constructor(elements = []) {
    this.addElements();
  }

  addElements(elements = []) {
    this.elements = {
      points: elements.filter(e => e.type === 'point'),
      lines: elements.filter(e => e.type === 'line')
    }
  }

  addPoint(x, y) {
    this.elements.points.push(new Point(x, y));
  }

  addLine(points=[]) { // [[x, y], [], [], ...]
    this.elements.lines.push(new Line(points));
  }

  toJSON() {
    return JSON.stringify(this.elements);
  }

  fromJSON(data="{ }", add=false) {
    let obj = JSON.parse(data);
    if(add) {
      this.elements.points =
this.elements.points.concat(obj.points||[]);
      this.elements.lines =
this.elements.lines.concat(obj.lines||[]);
    } else {
      this.elements = obj;
    }
  }

  deleteAll() {
    this.elements.points = [];
    this.elements.lines = [];
  }
}

let f = new Figure();
f.addPoint(10,20);
f.addPoint(10,10);
f.addLine([[10,20], [30,40], [50,60]]);
let json = f.toJSON();
console.log(json);

```

```
f.fromJSON(json, true);
console.log(f.elements.points.length);
console.log(f.elements.lines.length);
f.fromJSON('{"points":[{"type":"point","x":10,"y":20}, {"type":"point","x":10,"y":30}, {"type":"point","x":10,"y":-30}, {"type":"point","x":10,"y":20}, {"type":"point","x":20,"y":20}, {"type":"point","x":30,"y":20}, {"type":"point","x":130,"y":20}, {"type":"point","x":30,"y":20}, {"type":"point","x":0,"y":20}, {"type":"point","x":0,"y":-20}, {"type":"point","x":0,"y":20}], "lines":[{"type":"line", "points":[{"x":0,"y":0}, {"x":10,"y":0}, {"x":0,"y":10}, {"x":20,"y":0}, {"x":0,"y":20}], {"type":"line", "points":[{"x":30,"y":0}, {"x":10,"y":0}, {"x":0,"y":10}, {"x":20,"y":0}, {"x":0,"y":20}], {"type":"line", "points":[{"x":30,"y":0}, {"x":10,"y":-10}, {"x":0,"y":10}, {"x":20,"y":0}, {"x":0,"y":20}], {"type":"line", "points":[{"x":0,"y":0}, {"x":10,"y":0}, {"x":0,"y":10}, {"x":20,"y":0}, {"x":0,"y":20}]}]}');
console.log(f.elements.points.length);
console.log(f.elements.lines.length);
```

Figure 7. Program

In this program the classes are used to create objects of points and lines wherein it stores these objects to the Figure object which stores the x and y values. This data parsed into a JSON format and then output the values inside the figure elements of points and lines.

```
{
  "points": [
    {
      "type": "point",
      "x": 10,
      "y": 20
    },
    {
      "type": "point",
      "x": 10,
      "y": 10
    }
  ],
  "lines": [
    {
      "type": "line",
      "points": [
        {
          "type": "point",
          "x": 10,
          "y": 20
        },
        {
          "type": "point",
          "x": 30,
          "y": 40
        }
      ]
    },
    {
      "type": "point",
      "x": 50,
      "y": 60
    }
  ]
}
```

4 [challenge4.js:62](#)

2 [challenge4.js:64](#)

11 [challenge4.js:65](#)

4 [challenge4.js:67](#)

4 [challenge4.js:68](#)

› | **ctrl** **i** to turn on code suggestions. [Don't show again](#) **NEW**

Figure 8. Output

Based on this output the program is able to successfully extract the contents of the JSON data.

LAB Code Challenge #5

```
JavaScript
class Point {
  constructor(x, y) {
    this.type = 'point';
    this.x = x;
    this.y = y;
  }
}

class Line {
```



```

    constructor(points) { // [[x, y], [], [], ...]
      this.type = 'line';
      this.points = points.map(p => new Point(p[0], p[1]));
    }
  }

class Figure {

  constructor(elements = []) {
    this.elements = { points: [], lines: [] };
    this.addElements(elements);
  }

  static isSamePoint(point1, point2) {
    return point1 && point2 &&
      point1.x === point2.x &&
      point1.y === point2.y;
  }

  static isSameLine(line1, line2) {
    let retVal = line1 && line2 &&
      line1.points && line2.points &&
      line1.points.length === line2.points.length;

    if (retVal) {
      retVal = line1.points.every((p, i) => {
        let { x, y } = line2.points[i];
        return p.x === x && p.y === y;
      });
    }

    return retVal;
  }

  sortPoints() {
    this.elements.points = this.elements.points.sort((p1, p2) =>
      p2.x - p1.x == 0 ? p2.y - p1.y : p2.x - p1.x
    );
  }

  sortLines() {
    this.elements.lines = this.elements.lines.sort((l1, l2) => {
      let retVal = 0;

      for (let i = 0; i < l1.points.length; i++) {
        retVal = l2.points[i].x - l1.points[i].x == 0
          ? l2.points[i].y - l1.points[i].y
          : l2.points[i].x - l1.points[i].x;

        if (retVal) break;
      }
    });
  }
}

```

```

        }

        return retVal;
    });
}

cleanUp() {

    this.sortPoints();

    this.elements.points = this.elements.points.reduce(
        (p, c) =>
            Figure.isSamePoint(p[p.length - 1], c)
                ? p
                : (p.push(c), p),
        []
    );

    this.sortLines();

    this.elements.lines = this.elements.lines.reduce(
        (p, c) =>
            Figure.isSameLine(p[p.length - 1], c)
                ? p
                : (p.push(c), p),
        []
    );
}

addElement(elements = []) {

    this.elements.points = this.elements.points.concat(
        elements.filter(e => e.type === 'point')
    );

    this.elements.lines = this.elements.lines.concat(
        elements.filter(e => e.type === 'line')
    );

    this.cleanUp();
}

addPoint(x, y) {
    this.elements.points.push(new Point(x, y));
    this.cleanUp();
}

addLine(points = []) {
    this.elements.lines.push(new Line(points));
    this.cleanUp();
}

```

```

    }

    toJSON() {
        return JSON.stringify(this.elements);
    }

    fromJSON(data = "{}", add = false) {

        let obj = JSON.parse(data);

        if (add) {
            this.elements.points =
this.elements.points.concat(obj.points || []);
            this.elements.lines = this.elements.lines.concat(obj.lines
|| []);
        } else {
            this.elements = obj;
        }

        this.cleanUp();
    }

    deleteAll() {
        this.elements.points = [];
        this.elements.lines = [];
    }
}

// TEST

let f = new Figure();

f.addPoint(10, 20);
f.addPoint(10, 10);
f.addLine([[10, 20], [30, 40], [50, 60]]);

let json = f.toJSON();

console.log(json);

f.fromJSON(json, true);

console.log(f.elements.points.length);
console.log(f.elements.lines.length);

f.fromJSON('{"points":[{"type":"point","x":10,"y":20}, {"type":"point","x":10,"y":30}, {"type":"point","x":10,"y":-30}, {"type":"point","x":10,"y":20}, {"type":"point","x":20,"y":20}, {"type":"point","x":30,"y":20}, {"type":"point","x":130,"y":20}, {"type":"point","x":30,"y":20}, {"type":"point"

```

```
, "x":0, "y":20}, {"type":"point", "x":0, "y":-20}, {"type":"point", "x":0, "y":20}], "lines":[{"type":"line", "points":[{"x":0, "y":0}, {"x":10, "y":0}, {"x":0, "y":10}, {"x":20, "y":0}, {"x":0, "y":20}]}], {"type":"line", "points":[{"x":30, "y":0}, {"x":10, "y":0}, {"x":0, "y":10}, {"x":20, "y":0}, {"x":0, "y":20}]}], {"type":"line", "points":[{"x":30, "y":0}, {"x":10, "y":-10}, {"x":0, "y":10}, {"x":20, "y":0}, {"x":0, "y":20}]}], {"type":"line", "points":[{"x":0, "y":0}, {"x":10, "y":0}, {"x":0, "y":10}, {"x":20, "y":0}, {"x":0, "y":20}]}]}') );

console.log(f.elements.points.length);
console.log(f.elements.lines.length);
```

Figure 9. Program

In this program it updated the previous program wherein it now has the implementation of removing points and lines data on the Figure class. This is done by automatically sorts points and lines and removes duplicates whenever new elements are added or loaded from JSON. The program also includes static comparison methods to determine whether two points or two lines are identical.

```
{ "points": [{"type": "point", "x": 10, "y": 20}, {"type": "point", "x": 10, "y": 10}], "lines": [{"type": "line", "points": [{"type": "point", "x": 10, "y": 20}, {"type": "point", "x": 30, "y": 40}, {"type": "point", "x": 50, "y": 60}]}]} challenge5.js:147
2 challenge5.js:151
1 challenge5.js:152
8 challenge5.js:156
3 challenge5.js:157
```

Figure 10. Output

Based on the output of figure 10 the program was able to sort the data of point and lines successfully.

MODULE 4

LAB Code Challenge #1

```
JavaScript
class MyIterable {
  constructor() {
    this.data = [];
  }

  get length() {
    return this.data.length;
  }
}
```

```

has(n) {
    return this.data.includes(n);
}

add(n) {
    if(!this.has(n)) {
        this.data.push(n);
    }
}

del(n) {
    let index = this.data.indexOf(n);
    if(index !== -1) {
        this.data.splice(index, 1);
    }
}

[Symbol.iterator] = function* (){
    for(let index = 0; index<this.length; index++) {
        yield this.data[index];
    }
}

let iterable = new MyIterable();
iterable.add(2);
iterable.add(5);
iterable.add(3);
iterable.add(2);
iterable.del(3);

console.log(iterable.length); // -> 2
console.log(iterable.has(2)); // -> true
console.log(iterable.has(3)); // -> false
console.log(...iterable); // -> 2 5

```

Figure 11. Program

The program shown in figure 11 is able to generate an array of distinct numbers that may be iterated. The class offers methods to add, remove, check, and iterate over elements while internally storing values in an array.

2	challenge1.js:41
true	challenge1.js:42
false	challenge1.js:43
2 5	challenge1.js:44
Live reload enabled.	output.html:39

Figure 12. Output

Based on the output of figure 12 the program was able to run successfully.

LAB Code Challenge #2

JavaScript

```
let myDecorator = function(fn) {
  let cache = new Set();
  let included = function(...args) {
    let found = false;
    for(let e of cache) {
      let index = 0;
      console.log(`${e}`);
      found = true;
      for(let arg of args) {
        console.log(`    ${arg}`);
        if(arg !== e[index++]) {
          found = false;
          break;
        }
      }
    }
    if(found) {
      break;
    }
  }
  return found;
}

return function(...args) {
  if(included(...args)) {
    console.log(`arguments already used: ${args}`);
  } else {
    cache.add(args);
  }
  fn(...args);
}

}

let sum = function(...args) {
  let retVal = 0;
  for(let arg of args) {
    retVal += arg;
  }
  return retVal;
}

let dfn = myDecorator(sum);
dfn(2, 3, 4);
dfn(4, 5);
dfn(2, 3, 4); // -> arguments already used: 2,3,4
dfn(4, 5); // -> arguments already used: 4,5
```

Figure 13. Program

MyDecorator is a function defined by the program that accepts another function (fn) as an argument. Previously used parameter combinations are kept in a Set named cache inside the decorator. The program prints a message stating that the arguments were

already used if it finds the same set of arguments in the cache. The original function is carried out normally if the parameters are new; otherwise, they are saved in the cache.

2,3,4	challenge2.js:7
4	challenge2.js:10
2,3,4	challenge2.js:7
2	challenge2.js:10
3	challenge2.js:10
4	challenge2.js:10
arguments already used: 2,3,4	challenge2.js:24
2,3,4	challenge2.js:7
4	challenge2.js:10
4,5	challenge2.js:7
4	challenge2.js:10
5	challenge2.js:10
arguments already used: 4,5	challenge2.js:24

Figure 14. Output

In figure 14 it shows that the program was able to determine the different arguments being passed and output the correct item value.

LAB Code Challenge #3

```
let getPromiseArray = function(args) {
  let promises = args.map(arg => new Promise(function(resolve, reject){
    if(Number.isInteger(arg) && arg > 0) {
      let rnd = Math.random();
      setTimeout(resolve(arg), arg);
    } else {
      reject(new Error(`${arg} is not a positive integer`))
    }
  }));
  return promises;
}

let promises1 = getPromiseArray([10, 30, 5, 20, 'a']);
Promise.all(promises1).then(a => console.log(`all: ${a}`))
.catch(e => console.log(`all: ${e.message}`)); // -> any: 10
Promise.any(promises1).then(a => console.log(`any: ${a}`))
.catch(e => console.log(`any: ${e.message}`)); // -> all: a is not a positive integer
```

Figure 15. Program

In this program it uses a function that creates an array of promises based on a set of input values and then processes them. It evaluates the value and converts each value into a Promise. If the element is a positive integer, the promise resolves after a delay determined by the value itself using `setTimeout`. If the element is not a positive integer, the promise is rejected with an error message indicating that the value is invalid.

```
any: 10
all: a is not a positive integer
```

Figure 16. Output

Based on the output of figure 16 is that the program was able to properly evaluate the array.

LAB Code Challenge #4

JavaScript

```
let citiesCoords = {
  Berlin: { lat: 52.52, lon: 13.405 },
  Oslo: { lat: 59.9139, lon: 10.7522 },
  Yakutsk: { lat: 62.0355, lon: 129.6755 }
};

let getWeather = function(location, info) {
  let cities = Array.isArray(location) ? location : [location];
  const maxWindSpeed = 15;
  const minTemp = -20;

  let showWindInfo = function(weather){
    console.log(`WIND: ${weather.wind.speed} m/s,
    ${weather.wind.deg} deg`);
    if(weather.wind.speed > maxWindSpeed) {
      console.log(`WARNING! Wind speed over ${maxWindSpeed} m/s`);
    }
  };

  let showInfo = function(weather, type){
    console.log(`${type.toUpperCase()}: ${weather[type]} ${type ===
    'temp' ? 'C' : '%'} `);
    if(type === 'temp' && Number(weather.temp) < minTemp) {
      console.log(`WARNING! Temperature below ${minTemp}
      degrees`);
    }
  };

  cities.forEach(city => {
    let coords = citiesCoords[city];
    if(!coords) {
      console.log(`No coordinates found for ${city}`);
      return;
    }
    let url =
    `https://api.open-meteo.com/v1/forecast?latitude=${coords.lat}&longitude
    =${coords.lon}&current_weather=true`;
```



```

        fetch(url)
        .then(res => res.json())
        .then(data => {
            console.log(`\nCITY: ${city}`);
            let weather = {
                temp: data.current_weather.temperature,
                wind: {
                    speed: data.current_weather.windspeed,
                    deg: data.current_weather.winddirection
                },
                clouds: data.current_weather.cloudcover ?? 0,
                precipitation: data.current_weather.precipitation ??
0
            };

            if(info === 'all') {
                showWindInfo(weather);
                showInfo(weather, 'clouds');
                showInfo(weather, 'temp');
                showInfo(weather, 'precipitation');
            } else if(info === 'wind') {
                showWindInfo(weather);
            } else if(info) {
                showInfo(weather, info);
            }
        })
        .catch(e => console.log(e.message));
    });
};

let weather1 = getWeather('Berlin', 'wind');
// CITY: Berlin
// WIND: 16 m/s, 117 deg
// WARNING! Wind speed over 15 m/s

let weather2 = getWeather(['Oslo', 'Yakutsk'], 'all');
// CITY: Oslo
// WIND: 8 m/s, 170 deg
// CLOUDS: 0 %
// TEMP: 0 C
// PRECIPITATION: 0 %
//
// CITY: Yakutsk
// WIND: 0 m/s, 0 deg
// CLOUDS: 0 %
// TEMP: -40 C
// WARNING! Temperature below -20 degrees
// PRECIPITATION: 0 %

```

Figure 17. Program

In this program the `getWeather` function is designed to fetch and display current weather information for one or more cities. Since the Open-Meteo API requires latitude and longitude the function starts by mapping each city to its coordinates. The functions handle the display of wind data and other weather parameters like temperature, clouds, and precipitation, including issuing warnings if wind speed exceeds a defined maximum or if temperature falls below a minimum threshold.

CITY: Oslo	challenge4.js:13
WIND: 12.2 m/s, 67 deg	challenge4.js:13
CLOUDS: 0 %	challenge4.js:20
TEMP: 6.8 C	challenge4.js:20
PRECIPITATION: 0 %	challenge4.js:20
	challenge4.js:37
CITY: Yakutsk	challenge4.js:13
WIND: 4.5 m/s, 119 deg	challenge4.js:13
CLOUDS: 0 %	challenge4.js:20
TEMP: -21.1 C	challenge4.js:20
WARNING! Temperature below -20 degrees	challenge4.js:22
PRECIPITATION: 0 %	challenge4.js:20
	challenge4.js:37
CITY: Berlin	challenge4.js:13
WIND: 5.7 m/s, 145 deg	challenge4.js:13

Figure 18. Output

Based on the figure 18 the program was able to successfully fetch the data and print it accordingly.

LAB Code Challenge #5

JavaScript

```
const cityCoords = {
  Berlin: { lat: 52.52, lon: 13.405 },
  Oslo: { lat: 59.9139, lon: 10.7522 },
  Yakutsk: { lat: 62.0355, lon: 129.6755 }
};

let getWeather = async function(location, info) {
  const cities = Array.isArray(location) ? location : [location];
  const maxWindSpeed = 15;
  const minTemp = -20;

  const showWindInfo = function(weather) {
    if (weather.windspeed !== undefined) {
      const speed = weather.windspeed;
      const deg = weather.winddirection;
      console.log(`WIND: ${speed} m/s, ${deg} deg`);
      if (speed > maxWindSpeed) {
```

```

        console.log(`WARNING! Wind speed over ${maxWindSpeed}
m/s`);
    }
}

};

const showInfo = function(weather) {
    if (weather.temperature !== undefined) {
        console.log(`TEMP: ${weather.temperature} C`);
        if (weather.temperature < minTemp) {
            console.log(`WARNING! Temperature below ${minTemp}
degrees`);
        }
    }
};

const promises = cities.map(city => {
    const coords = cityCoords[city];
    if (!coords) {
        // Return a resolved promise with an error object
        return Promise.resolve({ city, error: `Coordinates for
${city} not found` });
    }

    const url =
`https://api.open-meteo.com/v1/forecast?latitude=${coords.lat}&longitude
=${coords.lon}&current_weather=true`;

    return fetch(url)
        .then(res => {
            if (!res.ok) throw new Error(`Failed to fetch weather
for ${city}`);
            return res.json();
        })
        .then(data => ({ city, weather: data.current_weather }))
        .catch(err => ({ city, error: err.message })); // Catch
fetch errors
    });

const results = await Promise.all(promises);

results.forEach(cityData => {
    console.log(`\nCITY: ${cityData.city}`);
    if (cityData.error) {
        console.log(`ERROR: ${cityData.error}`);
        return;
    }
    if (info === "wind" || info === "all")
showWindInfo(cityData.weather);

```

```

        if (info === "temp" || info === "all")
        showInfo(cityData.weather);
    });
};

// Example usage
getWeather('Berlin', 'wind');
getWeather(['Oslo', 'Yakutsk'], 'all');

```

Figure 19. Program

In this program it uses a weather API in which it is where we get the data which is similar to the previous program. By presenting the same asynchronous API calls and conditional warnings using more understandable language, this redesign basically shows how `async/await` may streamline promise-based operations without compromising functionality.

CITY: Berlin	challenge4.js:16
WIND: 5.4 m/s, 143 deg	challenge4.js:53
CITY: Oslo	challenge4.js:16
WIND: 12.2 m/s, 67 deg	challenge4.js:25
TEMP: 6.7 C	challenge4.js:53
CITY: Yakutsk	challenge4.js:16
WIND: 4.5 m/s, 119 deg	challenge4.js:25
TEMP: -21 C	challenge4.js:27
WARNING! Temperature below -20 degrees	

Figure 20. Output

Based on the output in Figure 18 the program is able to successfully fetch the data from the API and output each data for the city.

After conducting assignment 9.1, I concluded that I have learned Built-in Objects and Advanced Function Usage in JavaScript. For this assignment, we are to perform LAB code challenges in both modules 3 and 4 in JavaScript Essentials 2 in Netacad, which consist of five challenges on both modules. For this activity, I learned about how to effectively use built-in objects such as Arrays, Sets, Maps, and Math to simplify coding tasks. I also gained experience in creating and using higher-order functions, closures, and decorators to make my code more modular and reusable. Additionally, I practiced handling asynchronous operations using Promises and `async/await`, which helped me understand the flow of asynchronous JavaScript. This assignment also improved my debugging and problem-solving skills, as I had to carefully read requirements and test different scenarios. Overall, it strengthened my understanding of efficient coding practices and advanced features of JavaScript that are essential for real-world programming.

CONCLUSION

After performing this activity I was able to learn about the different built in objects and more advanced

function structure in Javascript. In this activity I was able to perform the different lab challenges wherein I was able to utilize my learning in using built in objects and advanced functions. In this activity I learned to use arrays and maps which are used as data handle for the class constructors. Also I was able to utilize asynchronous operations through Promises which are important for controlling the flow of the data in Javascript. Overall this activity helped me learn about more advanced techniques in using built in functions, classes in performing different tasks.